AGENTIC AI, LIVE

Building a Daily Planner With Claude Code

Guest Lecture Demo · Every step below actually ran

What Is Agentic AI?

- A chatbot answers once and stops. It has no idea if it was right.
- An agent plans a step, acts with real tools, checks the result, and repeats.
- Today: watch that loop build something real, end to end, live.

The Core Loop

1. Plan: break the goal into a concrete next step
2. Act: call a real tool (write code, run it, hit an API)
3. Observe: read back what actually happened
4. Verify: check the result against the real requirement
5. Repeat until it is genuinely done, not just attempted

What We're Building

› A daily planner from real constraints: prayer times, a class, seven flexible tasks

› A validator that enforces the constraints, not a script that just prints a plan

› Then: push it to Google Calendar, prep the class slides, draft a client reply

Step 1

Set the Rules

PROMPT: "Write constraints.json with today's fixed blocks and flexible tasks, plus validate.py to check for overlaps and enforce the fixed blocks."

Before generating anything, Claude writes down what 'correct' means as code. That turns a vague request into something it can actually check its own work against later.

ahsanul@secondbrain: ~/build

```
$ cat constraints.json
{
  "date": "2026-07-28",
  "day_window": {"start": "06:00", "end": "23:00"},
  "fixed_blocks": [
    {"name": "Fajr Prayer", "start": "06:00", "end": "06:20", "task_type": "prayer"},
    {"name": "Guest Lecture: Agentic AI (attend class)", "start": "11:00", "end": "12:00", "task_type": "class"},
    {"name": "Dhuhr Prayer", "start": "13:10", "end": "13:30", "task_type": "prayer"},
    {"name": "Asr Prayer", "start": "16:45", "end": "17:05", "task_type": "prayer"},
    {"name": "Maghrib Prayer", "start": "19:15", "end": "19:35", "task_type": "prayer"},
    {"name": "Isha Prayer", "start": "20:45", "end": "21:05", "task_type": "prayer"}
  ],
  "flexible_tasks": [
    {"name": "Prep slides for guest lecture", "duration_min": 45, "priority": "high", "must_be_before": "11:00", "task_
    {"name": "Reply to client email", "duration_min": 20, "priority": "high", "task_type": "email"},
    {"name": "Deep work: research reading", "duration_min": 120, "priority": "high", "task_type": "deep_work"},
    {"name": "Public speaking practice", "duration_min": 60, "priority": "high", "task_type": "skill"},
    {"name": "Startup deep-work block", "duration_min": 120, "priority": "medium", "hard_cap_min": 120, "task_type": "s
    {"name": "Gym", "duration_min": 60, "priority": "medium", "task_type": "health"},
    {"name": "Admin / inbox catch-up", "duration_min": 45, "priority": "low", "task_type": "admin"}
  ]
}
```

Step 2

First Draft

PROMPT: “Write schedule.py to generate today's schedule from constraints.json, then run it.”

Claude writes an honest first attempt: place fixed blocks, then slot flexible tasks in file order with no look-ahead. This is what a plain greedy scheduler actually produces, not a staged mistake.

ahsanul@secondbrain: ~/build

```
$ python3 -c "print today's timeline from schedule_v1.json"

  06:00-06:20 Fajr Prayer (fixed)
  11:00-12:00 Guest Lecture: Agentic AI (attend class) (fixed)
  13:10-13:30 Dhuhr Prayer (fixed)
  16:45-17:05 Asr Prayer (fixed)
  19:15-19:35 Maghrib Prayer (fixed)
  20:45-21:05 Isha Prayer (fixed)
  06:00-06:45 Prep slides for guest lecture
  06:45-07:05 Reply to client email
  07:05-09:05 Deep work: research reading
  09:05-10:05 Public speaking practice
  10:05-12:05 Startup deep-work block
  12:05-13:05 Gym
  13:05-13:50 Admin / inbox catch-up
```

Step 3

Observe the Failure

PROMPT: “Run validate.py against the generated schedule.”

Claude runs the same check a human reviewer would run. The output is not a final answer, it is a signal: three real overlaps, including one on top of Fajr prayer. This is the ‘observe’ step of the loop.

ahsanul@secondbrain: ~/build

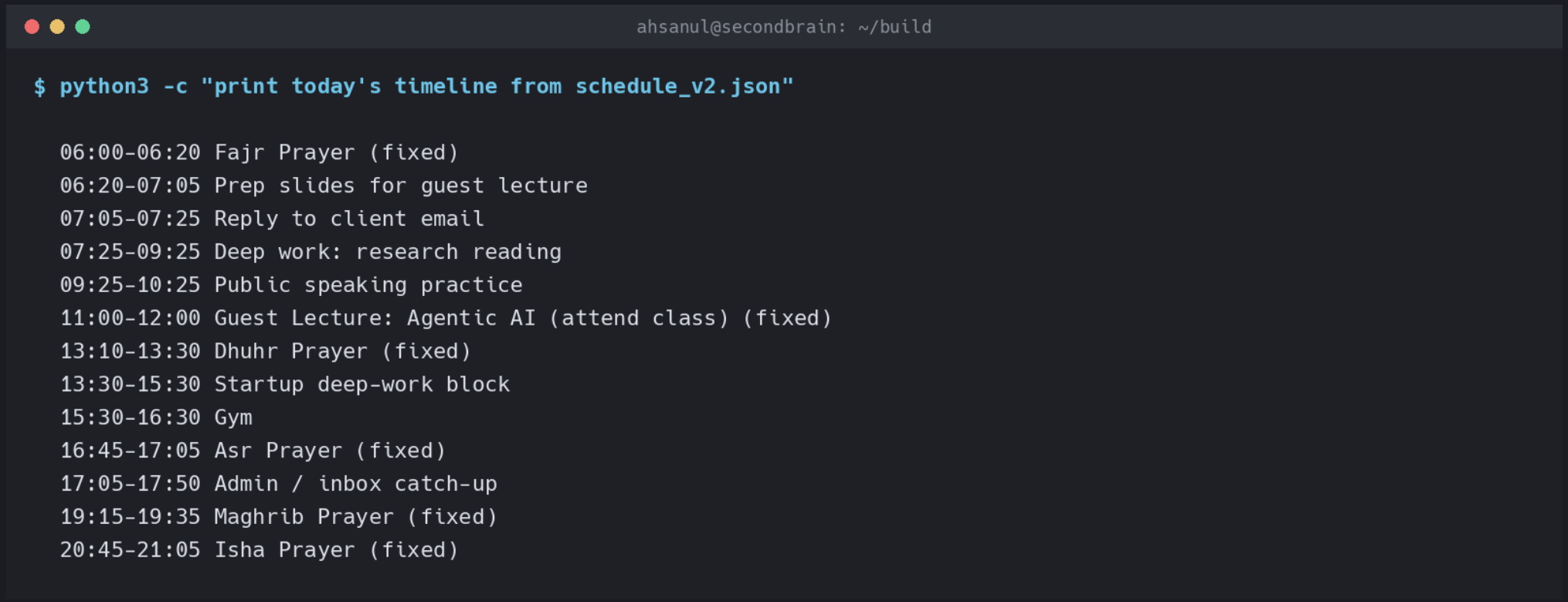
```
$ python3 validate.py schedule_v1.json
Validating schedule_v1.json against constraints.json ...

FAILED — 3 problem(s) found:

  x 'Prep slides for guest lecture' (360) overlaps 'Fajr Prayer' (ends 380) by 20 min
  x 'Guest Lecture: Agentic AI (attend class)' (660) overlaps 'Startup deep-work block' (ends 725) by 65 min
  x 'Dhuhr Prayer' (790) overlaps 'Admin / inbox catch-up' (ends 830) by 40 min
```

Step 4

Diagnose and Fix

PROMPT: “The validator failed. Diagnose why, fix schedule.py, then regenerate the schedule.”

Claude traces every failure to one root cause: no priority order and no look-ahead. The fix sorts tasks by priority and scans forward for a truly free slot, rather than patching each conflict by hand. That is judgment, not guesswork.

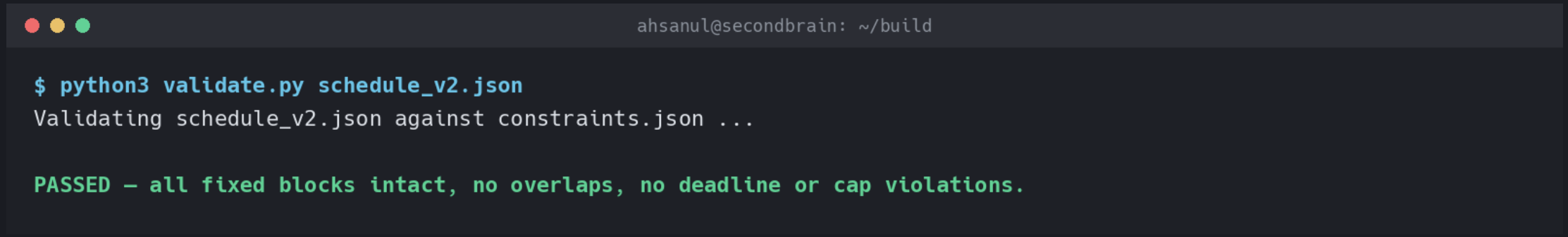


Step 5

Verify

PROMPT: “Re-run the validator against the fixed schedule.”

Only now does Claude call it done, because the same objective check that failed before now passes. The loop closes on evidence, not on confidence.

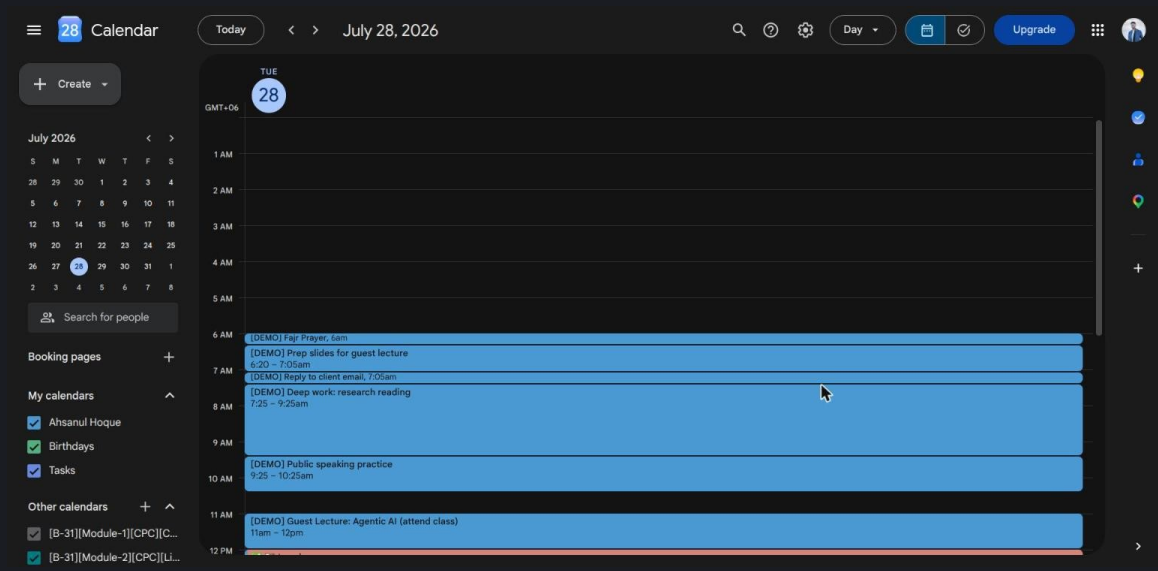


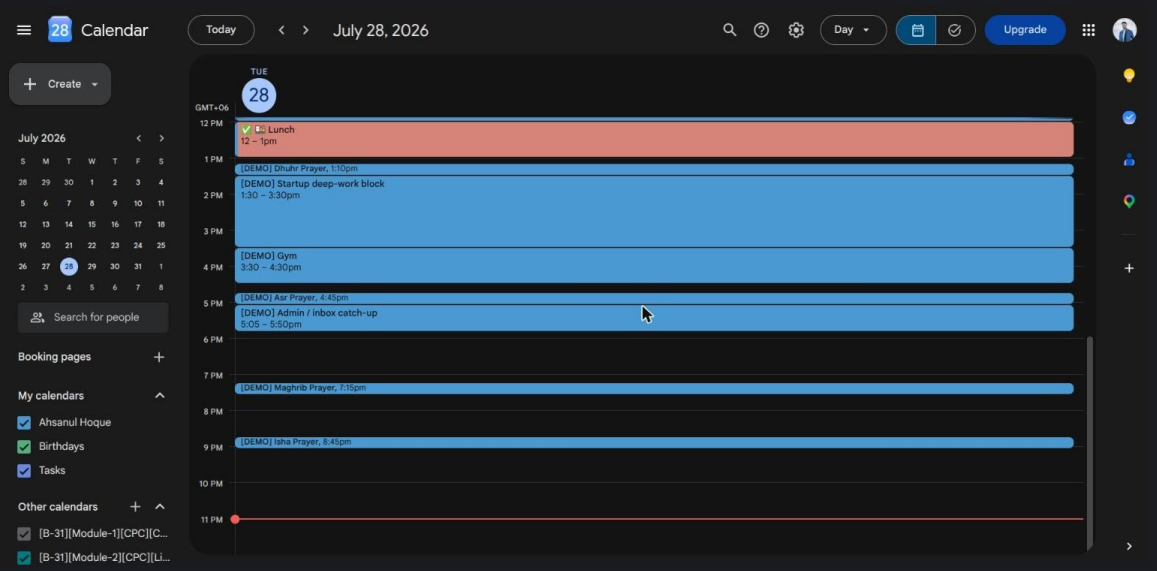
Step 6

Act in the Real World

PROMPT: “Push today's validated schedule to Google Calendar with reminders.”

Agentic does not stop at generating text. Claude calls a real calendar API, so the plan becomes something that will actually remind him at the right time.





Step 7

Execute a Task From Its Own Plan

PROMPT: “One of today's tasks is prepping slides for the guest lecture. Generate that slide deck.”

The plan is not a list Claude wrote and handed off. Claude carries out one of its own line items right there, as a real, checkable deliverable.

AGENTIC AI

What makes an AI agent ‘agentic’?

Guest Lecture · Built live with Claude Code

Today’s Live Demo

› Built a daily planner from constraints: prayer times, class, tasks

› First draft had a real scheduling conflict. The validator caught it.

› Diagnosed the bug, fixed the logic, re-validated until it passed

› Pushed the finished plan to Google Calendar with reminders

› Generated this exact slide deck as one of the plan’s own tasks

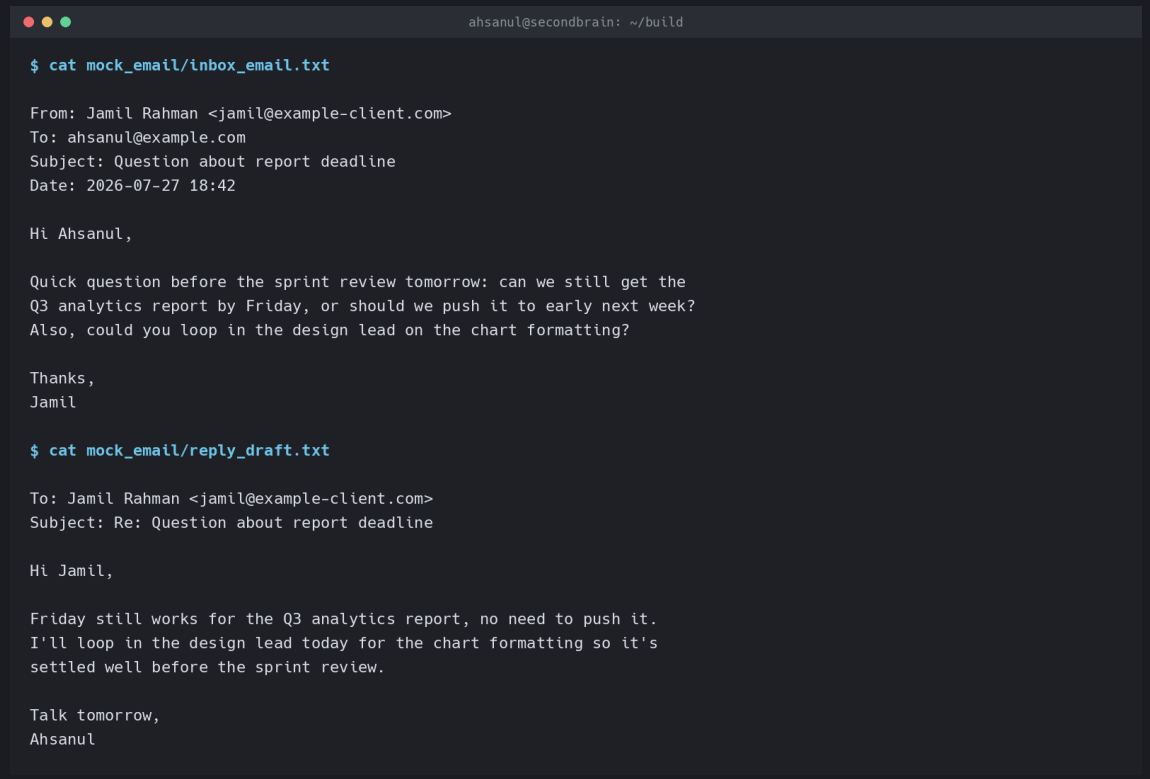
› Drafted a reply to a sample email, another task done autonomously

Step 8

Draft the Reply

PROMPT: “Another task is replying to a client email. Draft the reply.”

Same pattern again: read the actual request, act on it, and produce something the user only has to review, not write from scratch.



Key Takeaway

› Agentic = tool use + iteration + judgment under real constraints.

› Every step here was checked against something real: a validator, a calendar API, an actual task in the plan.

› Try it yourselves: give Claude Code a goal with a checkable definition of done, and watch what it does with the loop.